

CSE 344 – System Programming

Final Project

Sümeyye Zeynep Gürbüz — 200104004039

Project Report: Multi-threaded Distributed Chat and File Server

Overview

This project implements a robust, multi-threaded, TCP-based chat and file-sharing system using the client-server model in C, with a strong emphasis on concurrency, resource management, and synchronization. The server supports multiple simultaneous clients, group and private messaging, a room system, and queued file transfers, all with detailed logging and graceful error handling. The client provides an interactive command-line interface with color-coded feedback and live chat/file interaction.

1. Code Structure and Major Components

1.1. `common.h` — Shared Definitions

This header consolidates all shared constants, types, and utility functions for use across the client and server.

- **Constants:** Limits for buffer sizes, username/room/file name lengths, maximum clients/rooms/uploads, and queue capacity.
- **Data Structures:**
 - `client_t`: Client connection info (socket, username, room).
 - `room_t`: Room name and current capacity.

- `file_t`: Represents a file transfer (sender, receiver, file path, status, timestamps).
 - `queue_t`: Circular queue for file uploads, with thread synchronization primitives (mutex, semaphores) and upload state.
 - **Global Variables:**
 - `uploadQue`: The global file upload queue.
 - `loglock`: Mutex to serialize log file access.
 - **Utility Functions:**
 - `error`: Prints error message and exits.
 - `log_event`: Thread-safe, timestamped logging to `server.log`.
 - `write_fd`: Sends a string to a file descriptor and flushes.
-

1.2. Server Implementation

Main Server Flow

- **Initialization:** Sets up a TCP socket, binds to a port, and enters a listen loop.
- **Signal Handling:** Registers handlers for SIGINT/SIGTERM/SIGQUIT to ensure graceful shutdown (notifying clients, closing resources).
- **Client Connections:**
 - Accepts new clients (up to `MAX_CLIENTS`), spawning a detached thread for each with `client_handler`.
 - Ensures username uniqueness and input validation.
- **Global Resources:**
 - Arrays for active clients and rooms, protected by `clilock`.
- **Upload Queue:**
 - Initialized at startup.
 - Managed by up to `MAX_UPLOADS` worker threads, each running `upload_handler` to process enqueued file transfers.

Key Server Functions

- `client_handler`: Authenticates user, processes incoming commands, manages room membership, and handles disconnects.
- `handle_command`: Parses and executes chat/file commands such as `/join`, `/leave`, `/broadcast`, `/whisper`, `/sendfile`, `/help`, `/users`, `/rooms`, `/queue_status`, `/exit`.
- `handle_file_transfer`: Processes file transfer requests, reads files, reports queue wait times, and delivers files to receivers.
- `enqueue & dequeue`: Thread-safe operations on the upload queue with correct semaphore/mutex usage.
- `start_upload_handlers`: Spawns the worker threads managing the upload queue.
- `destroy_client`, `leave_room`: Cleanly remove client from all server structures, decrementing room counters and closing sockets.
- `cleanup_server_resources`: Ensures all threads, clients, rooms, and upload queue are safely freed and cleaned up on exit.

Thread Safety & Concurrency

- **Client and Room Arrays:** Protected using `clilock` to prevent concurrent modification.
- **Upload Queue:** Implements producer/consumer semantics with mutexes and semaphores for safe concurrent access and upload limiting.
- **Log File:** All log writes are mutex-protected to prevent race conditions.

File Transfer Queue

- **Queue-based Management:** Only up to 5 uploads are processed at once; overflow requests are queued.
- **File Validation:** Extension and size checks, path construction, and user existence checks.
- **User Feedback:** Notifies the sender of queue position and estimated wait; logs all file activity.

Graceful Shutdown

- **Signal Handlers:** On SIGINT/SIGTERM, notifies all clients, waits for uploads to finish, cleans all resources.
- **Resource Freeing:** Destroys all mutexes/semaphores, closes all sockets, and frees all allocated memory.

Logging

- **Event Logging:** Every connection, disconnection, room event, message, and file transfer is logged with a timestamp.
- **Sample Log Entry:**

```
[2025-05-18 14:02:31] [USER: alice12] - joined room 'project1'
```

1.3. Client Implementation

Main Client Flow

- **Startup:** Connects to the server using TCP with provided hostname and port.
- **Username Negotiation:** Prompts the user until a valid, unique username is accepted by the server.
- **Threaded I/O:**
 - **Sender Thread:** Reads from stdin, sends commands/messages to the server.
 - **Receiver Thread:** Reads server messages, prints them in color, handles server shutdown and disconnects.
- **Thread Safety:** Output and exit flags are mutex-protected.
- **Signal Handling:** On SIGINT, sets exit flag and closes resources gracefully.

Command-line Features

- **Supported Commands:**
 - `/join <room_name>`: Join/create a chat room.
 - `/leave`: Leave current room.
 - `/broadcast <msg>`: Send message to all users in current room.

- `/whisper <username> <msg>`: Send private message.
 - `/sendfile <filename> <username>`: Request file transfer to user (with validation).
 - `/help, /users, /rooms, /queue_status`: Get info on available commands, users, rooms, upload queue.
 - `/exit`: Disconnect from server.
 - **Color-coded Feedback**: Errors, warnings, successes, and info are color-highlighted for clarity.
 - **Clean Exit**: Ensures all threads and mutexes are destroyed, and server is notified on exit.
-

2. Features & Functional Highlights

2.1. Multi-threaded Concurrency

- Server handles many clients simultaneously.
- Upload processing is distributed among multiple worker threads.
- All shared state is protected by mutexes or semaphores to avoid race conditions.

2.2. Room and User System

- **Rooms**: Dynamically created on join, with per-room capacity limits and auto-deletion when empty.
- **Users**: Unique, validated usernames; ability to list users and rooms at any time.

2.3. File Sharing

- **Upload Queue**: Simulates limited server resources, queues excess upload requests, and notifies users of queue status and wait times.
- **Supported File Types**: `.txt`, `.pdf`, `.png`, `.jpg`.
- **Per-file Validation**: Size capped at 3MB, valid extensions, and path checks.

2.4. Logging & Error Handling

- **Server Logs**: Every important user and server event is logged with timestamps.
 - **Input Validation**: Usernames, commands, room names, file names/extensions, and sizes are validated; detailed error messages are provided.
 - **Server and Client Robustness**: Handles disconnects, invalid input, full server/rooms, and OS signals gracefully.
-

3. Key Implementation Snippets & Data Flow

3.1. File Upload Queue Logic

- **Enqueue:** Checks if queue is full, notifies sender, places the request in the queue, and logs the event.
- **Dequeue & Processing:** Worker threads wait for items, process uploads, update current upload count, and notify both sender and receiver of status.

3.2. Command Parsing

- **On Client:** Reads user input, sends raw strings to server.
- **On Server:** Checks command prefix, parses arguments, validates, executes appropriate action, and returns feedback/logs.

3.3. Room Management

- When a user joins a room:
 1. Checks if room exists.
 2. If not, creates a new room if below max limit.
 3. Updates room's user count.
 4. Removes user from old room if needed, deleting old room if empty.

3.4. Graceful Shutdown

- **On Signal:** All clients receive a shutdown message, all threads are joined/cancelled, all resources are freed, and log entry is made.

4. Example Log Entries

```
[2025-05-18 14:02:31] [USER: alice12] - joined room 'project1'
[2025-05-18 14:03:01] [LOGIN] user 'john45' connected from 192.168.1.44
[2025-05-18 14:03:05] [JOIN] user 'john45' joined room 'team1'
[2025-05-18 14:03:12] [BROADCAST] user 'john45': Hello team!
[2025-05-18 14:03:22] [SEND FILE] 'project.pdf' sent from john45 to alice99 (success)
```

5. Summary of Main Components

- **common.h:** Project-wide types, limits, and thread-safe utility functions.
- **Server Source:** Socket and resource initialization, client and room management, command processing, file upload queueing, synchronization, shutdown, and logging.
- **Client Source:** Connection and authentication, multi-threaded interaction, color-coded CLI, and robust handling of disconnects and signals.